EEE3096S Practical 2
Memory-Mapped I/O and Hardware SPI

Adam Foflonker (FFLADA001)
Sibahle Ntando Mndebele (MNDSIB016)

2nd October 2026

1 Memory-Mapped GPIO

PC13 was driven as a digital output using explicit `volatile` pointers built from base address plus register offset, with no HAL GPIO calls.

Addresses and offsets

Table 1: Task 1 register addresses ($A_{\text{register}} = B + O$).

Register	Base B	Offset O	Address	Source
`RCC_AHBENR`	`0x4002 1000`	`0x14`	`0x4002 1014`	RM0091 §7.4.6
`GPIOC_MODER`	`0x4800 0800`	`0x00`	`0x4800 0800`	RM0091 §9.4.1
`GPIOC_ODR`	`0x4800 0800`	`0x14`	`0x4800 0814`	RM0091 §9.4.6
`GPIOC_BSRR`	`0x4800 0800`	`0x18`	`0x4800 0818`	RM0091 §9.4.7
`GPIOC_BRR`	`0x4800 0800`	`0x28`	`0x4800 0828`	RM0091 §9.4.11

All peripheral base addresses are from RM0091 Table 1 (*STM32F0xx peripheral register boundary addresses*); the register offsets are from the individual register descriptions in RM0091 §7.4 and §9.4.

Bit masks

Table 2: Bit masks used for PC13.

Register	Field	Mask / value	Purpose
`RCC_AHBENR`	IOPCEN (bit 19)	`0x0008 0000`	enable GPIOC clock
`GPIOC_MODER`	MODER13 [27:26]	clear `0x0C00 0000`, set `0x0400 0000`	general-purpose output (01)
`GPIOC_BSRR`	BS13 (bit 13)	`0x0000 2000`	set PC13 high
`GPIOC_BSRR`	BR13 (bit 29)	`0x2000 0000`	set PC13 low
`GPIOC_BRR`	BR13 (bit 13)	`0x0000 2000`	initial low level

Why `volatile`

Peripheral registers can change without any action by the program (status flags, input data) and writes to them have side effects. `volatile` forces the compiler to perform every read and write exactly as written, so it cannot cache a register value in a CPU register, hoist a read out of a loop, or delete a write it believes is redundant.

Measurement

2 Hardware SPI Configuration

SCK calculation

$$f_{\text{SCK}} = \frac{f_{\text{PCLK1}}}{2^{\,BR+1}} = \frac{8\,\text{MHz}}{2^5} = \frac{8\,000\,000}{32} = 250\,\text{kHz}$$

BR = 3 would give 500 kHz, so BR = 4 is the divider that gives 250 kHz.

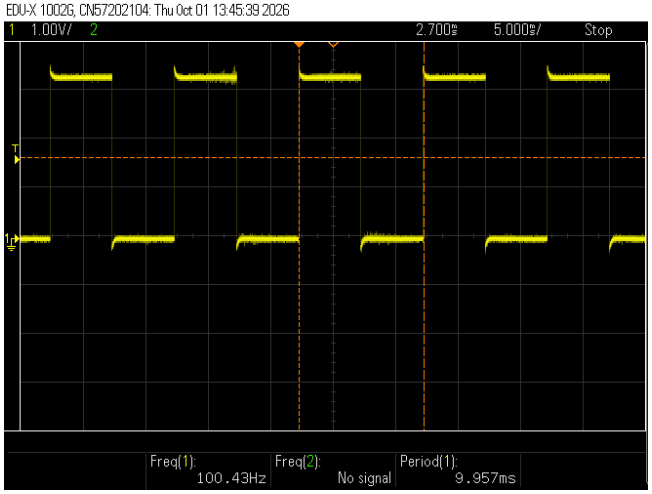


Figure 1: PC13 output (CH1, 1 V/div, 5 ms/div). High level ≈ 3.3 V and low level ≈ 0 V (read from the graticule); measured period 9.957 ms (100.43 Hz). The half-period is set to 5 ms in code, so the nominal period is 10 ms; the difference is 0.43 %.

Table 3: Task 2 SPI configuration.

Item	Value
SPI instance	SPI2
Peripheral clock	8 MHz (PCLK1)
PB13 (SCK) AF	AF0
PB14 (MISO) AF	AF0
PB15 (MOSI) AF	AF0
Clock divider	BR $= 100_2$, /32
Predicted SCK	250 kHz
CPOL	0
CPHA	0
Bit order	MSB first (LSBFIRST $= 0$)

Other configuration decisions

- **Clocks:** GPIOB (`RCC_AHBENR`, bit 18) and SPI2 (`RCC_APB1ENR`, bit 14).
- **PB12 (CS):** output, ODR set high *before* switching the pin to output mode so CS never glitches low. CS is active low, so it is held high to keep the EEPROM deselected.
- **Software NSS:** SSM = 1, SSI = 1. CS is driven from a GPIO, so without this the peripheral can see NSS low, raise a mode fault and clear MSTR.
- **CR2:** DS = 0111 (8-bit), FRXTH = 1 so RXNE is set after one byte.
- **MISO pull-up:** the EEPROM SO pin is high-impedance while CS is high, so a pull-up stops MISO floating.
- **Alternate-function assignment:** PB13/PB14/PB15 map to SPI2_SCK/MISO/MOSI respectively on AF0, per STM32F051 datasheet Table 15 (alternate functions for port B).

Register readback

Table 4: Register values read in the debugger (`RUN_TASK 2`).

Register	Expected	Read on board
`SPI2_CR1`	`0x0364`	`0x0364`
`SPI2_CR2`	`0x1700`	`0x1700`
`SPI2_SR`	`0x0002`	`0x0002`
`GPIOB_MODER`	`0xA9000280`	`0xA9000280`
`GPIOB_AFRH`	`0x00000000`	`0x00000000`

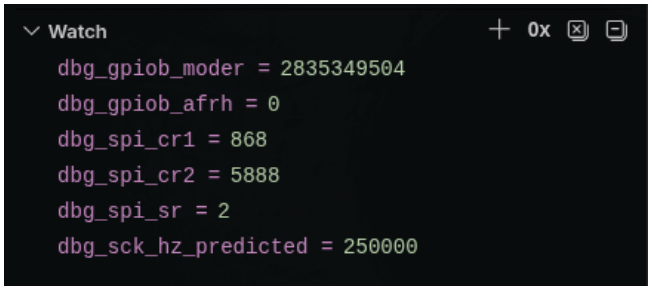


Figure 2: Debugger view of SPI2 and GPIOB registers.

3 SPI Waveform

Group test byte

Student numbers: $n_1 = 1$ (FFLADA001), $n_2 = 16$ (MNDSIB016).

$$n_1 \oplus n_2 = 00000001_2 \oplus 00010000_2 = 00010001_2 = \texttt{0x11}$$
$$B = (\texttt{0x11} + \texttt{0x3D}) \bmod 256 = \texttt{0x4E}$$

$B \neq \texttt{0x00}, \texttt{0xFF}$, so no correction is needed.

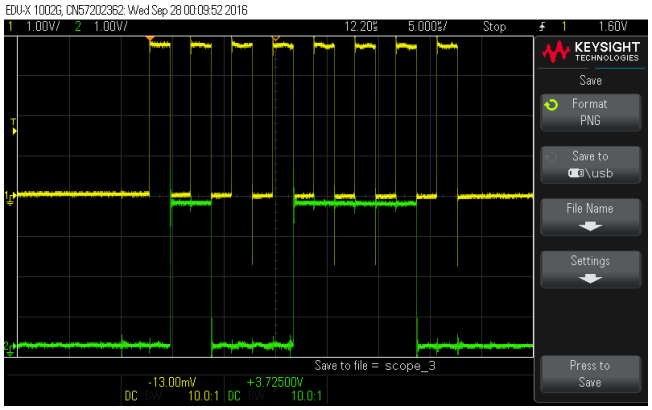


Figure 3: SCK (CH1) and MOSI (CH2) for one byte, CS held high (1 V/div, 5 μs/div). Cursors: X1 = -100 ns on the rising edge of pulse 1, X2 = 28.0 μs on the rising edge of pulse 8, $\Delta X = 28.1\,\mu$s.

Capture

Measured SCK

- The cursors span 7 clock periods (rising edge of pulse 1 to rising edge of pulse 8): $\Delta X = 28.1\,\mu$s, so $T_{\text{SCK,measured}} = 28.1/7 = 4.014\,\mu$s.
- $f_{\text{SCK,measured}} = 1/T = 249.1\,\text{kHz}$.
- $\%\text{error} = \dfrac{|249.1 - 250|}{250} \times 100 = 0.36\,\%$.
- Idle state: low. Clock pulses counted: 8.

Manual decode

Table 5: MOSI level at each SCK rising edge, read from Fig. 3.

Clock pulse	1	2	3	4	5	6	7	8
Bit position	b7	b6	b5	b4	b3	b2	b1	b0
MOSI level	0	1	0	0	1	1	1	0

Decoded bits = $0100\,1110_2$ = `0x4E`. MSB is transmitted first. Each bit is read at the SCK rising edge; MOSI changes only on falling edges, so the data is stable on the sampling edge (CPOL = 0, CPHA = 0). The EEPROM latches SI on the rising edge [1].

4 EEPROM Driver

Group address

$$A = (n_1 + 3n_2) \bmod 256 = (1 + 48) \bmod 256 = 49 = \texttt{0x31}$$

Address bytes on the fitted part: 2, MSB first. Confirmed against the CAT25040 datasheet: the READ and WRITE instructions are followed by `A7`–`A0`, and the part is organised as 512×8; the waveform decode in Fig. 3 also shows that data is shifted out MSB first.

Table 6: EEPROM transactions (opcodes from the CAT25040 datasheet, Instruction Set table).

Operation	Sequence (CS low → CS high)	SCK pulses
Write enable	`0x06`	8
Write byte	`0x02`, `0x00`, `0x31`, `0x4E`	32
Poll status	`0x05`, dummy byte (repeat until RDY = 0)	16
Read byte	`0x03`, `0x00`, `0x31`, dummy byte	32

Transaction sequences

The write-enable latch is set only on the CS rising edge, so WREN is its own transaction. The internal write cycle starts on the CS rising edge after the data byte.

Write-completion mechanism

After the write, RDSR is issued repeatedly and bit 0 (RDY) is tested: RDY = 1 means a write cycle is in progress, RDY = 0 means ready (CAT25040 Status Register table). No fixed delay is used. A timeout of 50 ms is only a hang guard. Measured write time: `eeprom_write_wait_ms` = 4 ms (datasheet maximum $t_{WC} = 5$ ms, AC Characteristics, Write Cycle Time of [1]).

Result

Table 7: Write/read-back result (`RUN_TASK 4` debugger capture).

Item	Value
Status before write (`eeprom_status_before`)	`0x00`
Status after write (`eeprom_status_after`)	`0x00`
Value written to address `0x31`	`0x4E`
Value read back (`eeprom_read_value`)	`0x4E`
Verify OK (`eeprom_verify_ok`)	`0x01`
Write time (`eeprom_write_wait_ms`)	4 ms
Timeout count (`eeprom_timeout_count`)	`0x00`
LED pattern (LED7..LED0)	`0100 1110`
Persistence after reset/power cycle	`PASS`

5 Fault Diagnosis

Diagnosis based on our own bench measurements.

- **First meaningful difference:** the SCK frequency, idle level and pulse count are identical in both captures. The only configuration difference is one bit in `SPI2_CR1`: `dbg_spi_cr1 = 0x0365` in the faulty build versus `0x0364` in the working build. The differing bit is bit 0, CPHA. On the scope, the MOSI edges consequently move from the SCK falling edges (Fig. 5 (right)) to the SCK rising edges (Fig. 5 (left)), a shift of half a bit period (2 μs).
- **Register field responsible:** `SPI2_CR1`, CPHA (bit 0): faulty value 1 → corrected value 0 (RM0091 §27.7.1, SPI control register 1).
- **Why the transaction failed electrically:** with CPHA = 1 the master changes MOSI on the same rising edge on which the EEPROM latches SI [1]. The data-setup time t_{SU} before that

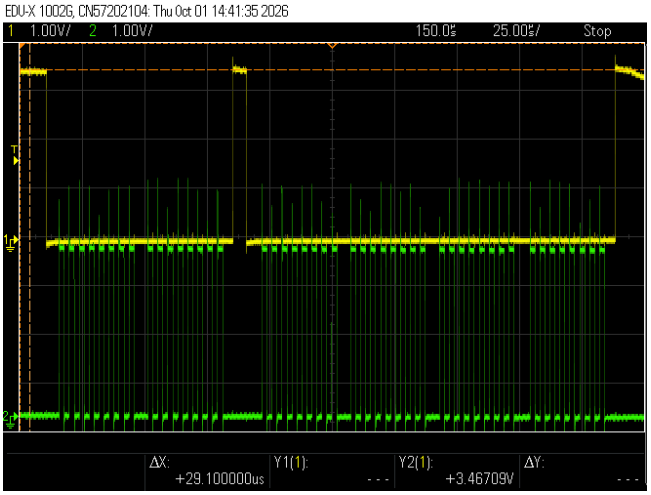


Figure 4: CS (CH1) and SCK (CH2) (1 V/div, 25 μs/div): a status read (RDSR) followed by a data read (READ). CS is low for about 75 μs during the first transaction (16 clock pulses) and about 149 μs during the second (32 clock pulses); SCK is idle low and only toggles while CS is low.

Table 8: Expected-versus-observed diagnosis.

Expected	Observed	Cause	Correction
MOSI changes on SCK falling edges and is stable at each rising edge (SPI mode 0, CPOL = 0, CPHA = 0), because the EEPROM latches SI on the rising edge [1].	SCK is normal (idle low, 4 μs period, 8 pulses per byte), but MOSI transitions coincide with the SCK *rising* edges (Fig. 5 (left)). Registers: `dbg_spi_cr1` = `0x0365`, `dbg_spi_sr` = `0x0002`, `dbg_sck_hz_predicted` = `0x3D090`.	`SPI2_CR1` CPHA (bit 0) = 1, i.e. SPI mode 1, instead of 0. Confirmed in the debugger: `dbg_spi_cr1` read `0x0365` instead of `0x0364` — a one-bit difference.	Clear CPHA (`SPI2_CR1` bit 0 = 0) while SPE is still 0, which is the `RUN_TASK 4` configuration (Fig. 5 (right)).

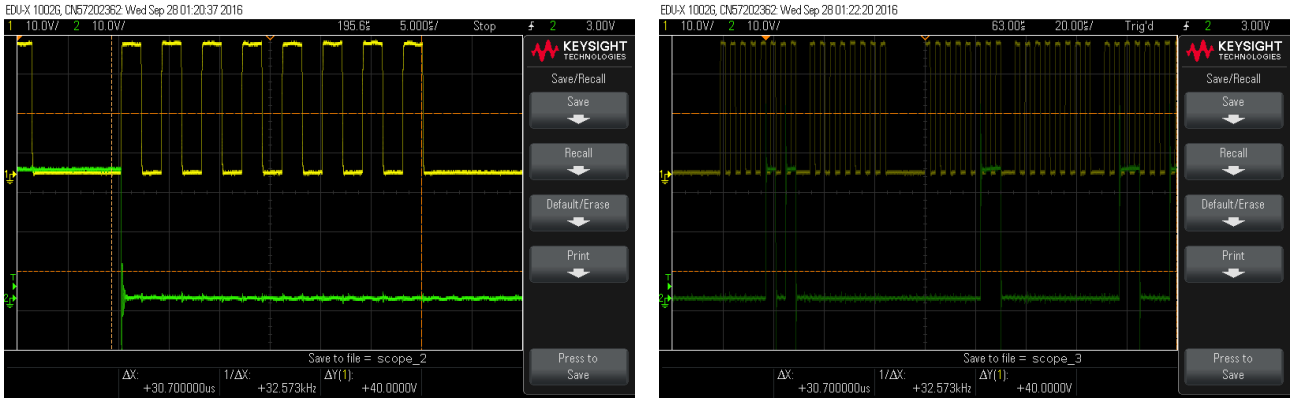


Figure 5: Left: faulty case (`RUN_TASK 5`, CPHA = 1). Right: corrected case (`RUN_TASK 4`, CPHA = 0). Both show SCK (CH1) and MOSI (CH2) at 1 V/div and 5 μs/div. In the faulty capture MOSI edges line up with SCK rising edges; in the corrected capture they line up with SCK falling edges.

edge (5–20 ns depending on supply, AC Characteristics of [1]) is therefore not met, so each bit is captured while it is still changing and the instruction, address and data bytes are corrupted. An invalid op-code is ignored by the EEPROM. In the other direction, the master samples MISO on the falling edge, the same edge on which the EEPROM shifts SO out (valid only after t_V), so read data is also unreliable.

- **Why the correction works:** with CPHA = 0 MOSI is set up on the falling edge and is stable for half a clock period (2 μs, far above t_{SU} and t_H) before the rising edge at which the EEPROM latches it, and the master samples MISO on the rising edge, half a period after the EEPROM shifted SO out.

6 Non-Blocking FSM

State diagram

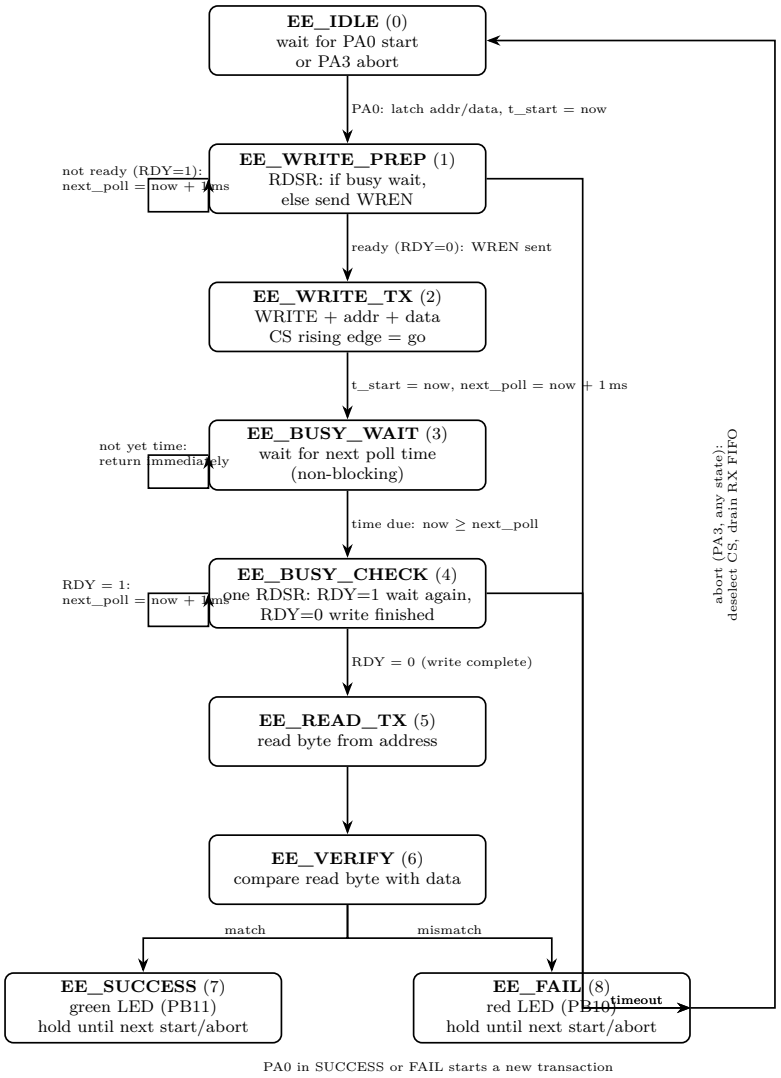


Figure 6: Final non-blocking state machine (state numbers match `ee_state` in the debugger).

Transition logic

- **IDLE → WRITE PREP:** PA0 edge. Address and data are latched from `eeprom_test_addr` and `eeprom_test_byte`.

- **WRITE PREP:** one RDSR; if RDY = 1 re-check after 1 ms, otherwise send WREN and go to WRITE TX.
- **WRITE TX → BUSY WAIT:** WRITE command, address and data sent, CS raised.
- **BUSY WAIT ↔ BUSY CHECK:** wait until the next poll time (tick comparison, returns immediately), then one RDSR; RDY = 1 returns to BUSY WAIT, RDY = 0 goes to READ TX. If RDY has not cleared after 50 ms the FSM goes to FAIL.
- **READ TX → VERIFY → SUCCESS/FAIL:** read back, compare with written byte; a mismatch goes to FAIL.
- **Abort:** PA3 deselects the EEPROM (CS high), drains the SPI receive FIFO, and returns to IDLE.
- **Outputs:** PB0–PB7 show the last byte read; PB11 green on success, PB10 red on failure.

Why it is non-blocking

The 5 ms EEPROM write cycle is never waited on in a blocking loop. In `EE_BUSY_WAIT` the FSM only compares `HAL_GetTick()` with `fsm_next_poll` and returns immediately. Every other active state performs a single short SPI transaction; a 32-clock transaction takes 128 μs at 250 kHz, and a 16-clock status read takes 64 μs. PA3 is sampled on every pass of the main loop.

References

[1] ON Semiconductor, *CAT25010/20/40 1-Kb, 2-Kb and 4-Kb SPI Serial CMOS EEPROM Datasheet*, Doc. No. CAT25010/D.

[2] STMicroelectronics, *RM0091: STM32F0x1/STM32F0x2/STM32F0x8 advanced ARM-based 32-bit MCUs Reference Manual*.

[3] STMicroelectronics, *STM32F051x4/x6/x8 Datasheet*.